

Network Packet Sniffer using Scapy in Python

Cybersecurity and Networking Mini Project Report

This project focuses on building a real-time Network Packet Sniffer using Python and the Scapy library. The system captures live network packets from the network interface, analyzes the packet structure layer-by-layer, and detects suspicious or anomalous traffic. This project helps students understand networking protocols, packet structures, cyber threats, and traffic monitoring techniques used in cybersecurity.

1. Project Objectives

- Capture network packets in real-time.
- Analyze protocols such as TCP, UDP, ICMP, HTTP, and DNS.
- Display source IP, destination IP, ports, and protocol details.
- Detect suspicious traffic patterns and anomalies.
- Store logs for further investigation and monitoring.
- Learn networking concepts and packet dissection techniques.

2. Technologies Used

- Python Programming Language
- Scapy Library
- Wireshark (Optional for verification)
- Operating System: Linux / Windows
- Networking Concepts and Protocols

3. System Requirements

Hardware Requirements:

- Minimum 4 GB RAM
- Intel i3 or above processor
- Stable internet/network connection

Software Requirements:

- Python 3.x
- Scapy Library
- VS Code / PyCharm
- Npcap (for Windows packet capturing)

4. Working of the Project

The packet sniffer continuously monitors incoming and outgoing packets from the selected network interface. Whenever a packet is captured, the program extracts details such as source IP address, destination IP address, protocol type, MAC address, and payload data.

The Scapy library dissects packets into layers such as:

- Ethernet Layer

- IP Layer
- TCP/UDP Layer
- Application Layer

The system also checks for abnormal traffic patterns like:

- Port scanning attempts

- Suspicious IP addresses
- High-frequency packet transmission
- Unknown protocol activities

5. Features of the Packet Sniffer

- Real-time packet monitoring
- Protocol identification
- Packet filtering support
- Anomaly detection logging
- Layer-by-layer packet analysis
- Lightweight and easy-to-use interface

6. Sample Python Code

```
from scapy.all import *

def packet_callback(packet):
    if packet.haslayer(IP):
        print("Source:", packet[IP].src)
        print("Destination:", packet[IP].dst)
        print("Protocol:", packet.summary())
```

```
sniff(prn=packet_callback, store=0)
```

7. Advantages

- Helps in understanding real-time networking.
- Useful for cybersecurity learning and monitoring.
- Detects suspicious traffic activity.
- Provides hands-on packet analysis experience.
- Enhances ethical hacking and network security skills.

8. Limitations

- Requires administrator/root privileges.
- Cannot decrypt encrypted traffic automatically.
- High traffic networks may produce huge logs.
- Ethical and legal permission is required before monitoring networks.

9. Future Enhancements

- Add graphical dashboard visualization.
- Integrate machine learning for anomaly detection.
- Export logs into CSV or database.
- Create alert notifications for suspicious traffic.
- Add protocol-based filtering and reporting.

10. Conclusion

The Network Packet Sniffer project is an effective cybersecurity and networking tool developed using Python and Scapy. It provides practical knowledge of network communication, packet structures, and traffic monitoring. This project is highly useful for students learning cybersecurity, ethical hacking, digital forensics, and network administration.

11. Common Protocols Analyzed

Protocol	Purpose	Common Use
TCP	Reliable communication	Web browsing, Email
UDP	Fast transmission	Streaming, Gaming
ICMP	Error reporting	Ping requests
HTTP	Web communication	Accessing websites

DNS	Domain resolution	Website lookup
-----	-------------------	----------------

Prepared For: Cybersecurity / Networking Academic Project